

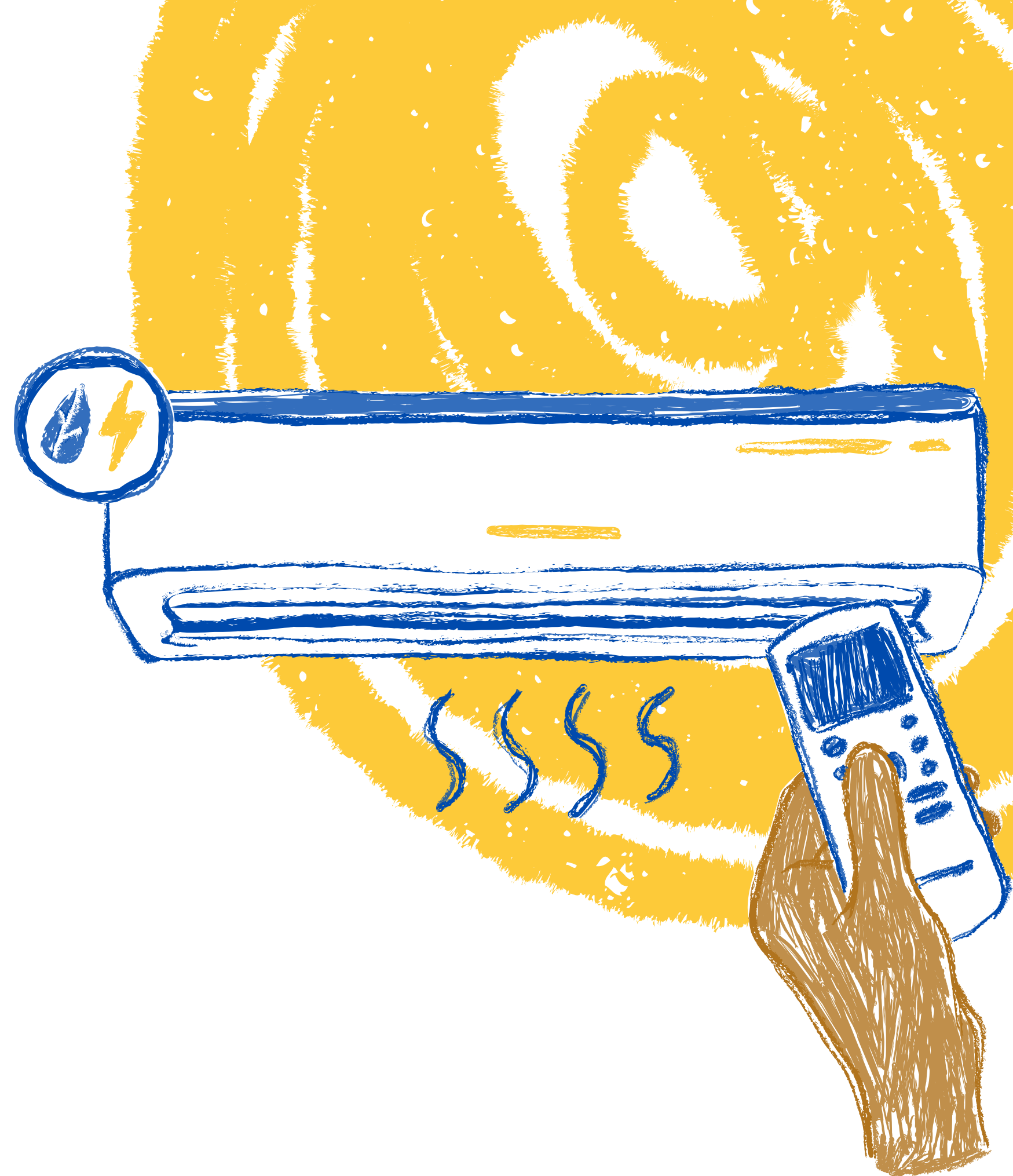
Sistema Didáctico de Control Térmico con ESP32

Presentado por:

Marlyn Nathalia Mora Riascos. *Nathalia*
Ever Daniel Hernández Tulcán
Marilyn Melissa Micanquer Laguna

Presentado a:

Jaime E. Arango Castro



ARQUITECTURA GENERAL

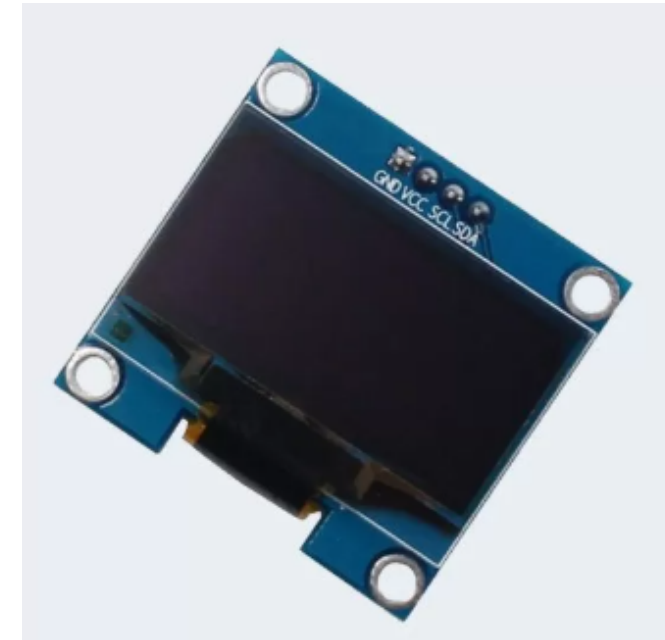
¿Qué problema se busca resolver?

Mantener la temperatura de un sistema dentro de un rango adecuado mediante la medición continua de temperatura y el accionamiento automático de un sistema de calefacción y ventilación.

ESP 32



pantalla ssd1316



Bombillo calefactor AC



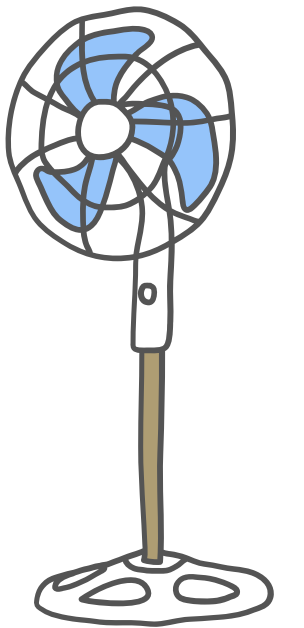
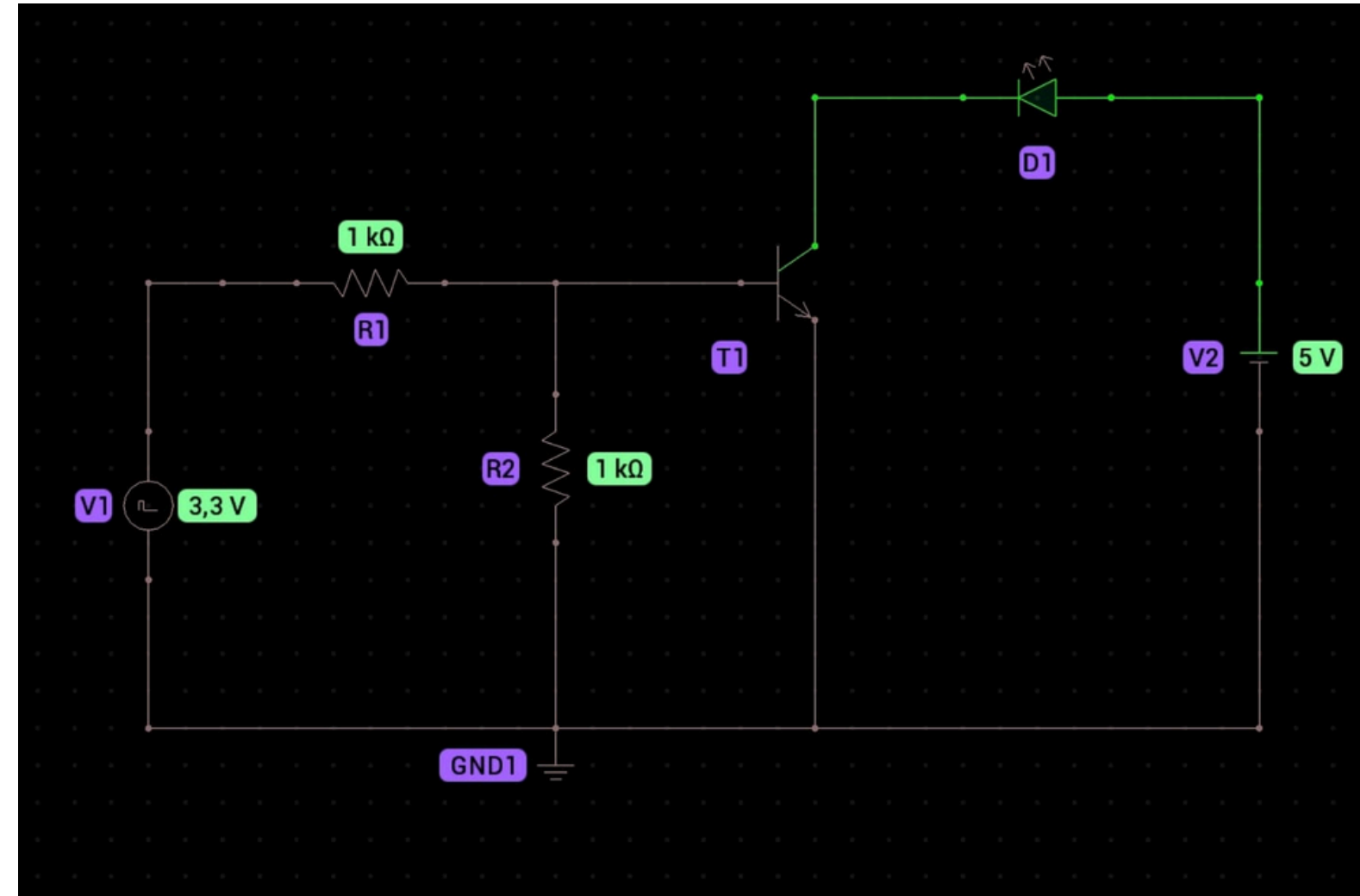
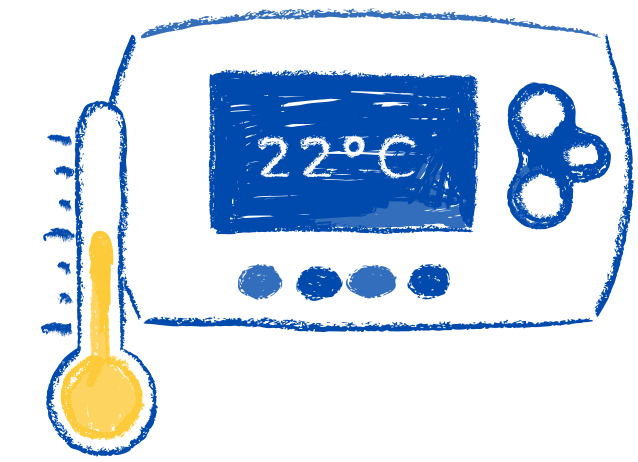
ventilador



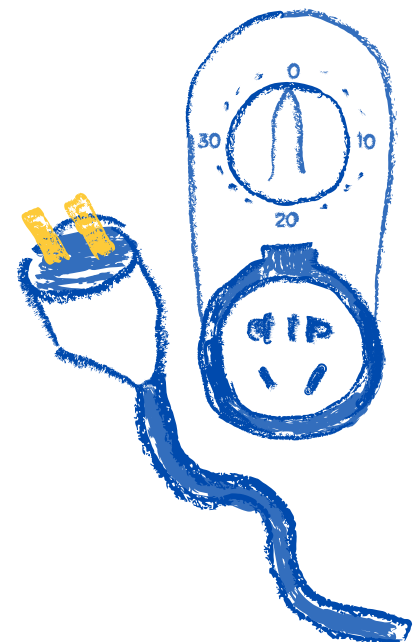
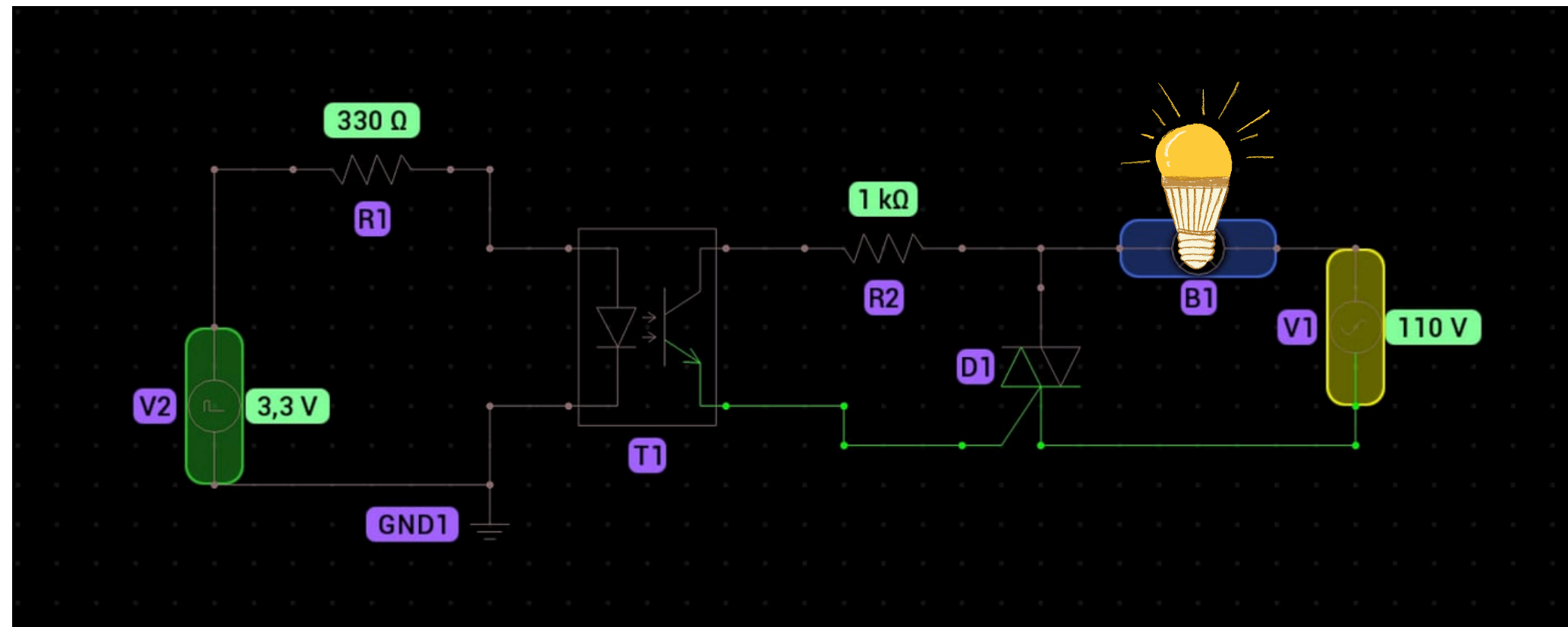
Termistor



simulación Control PWM de ventilador DC

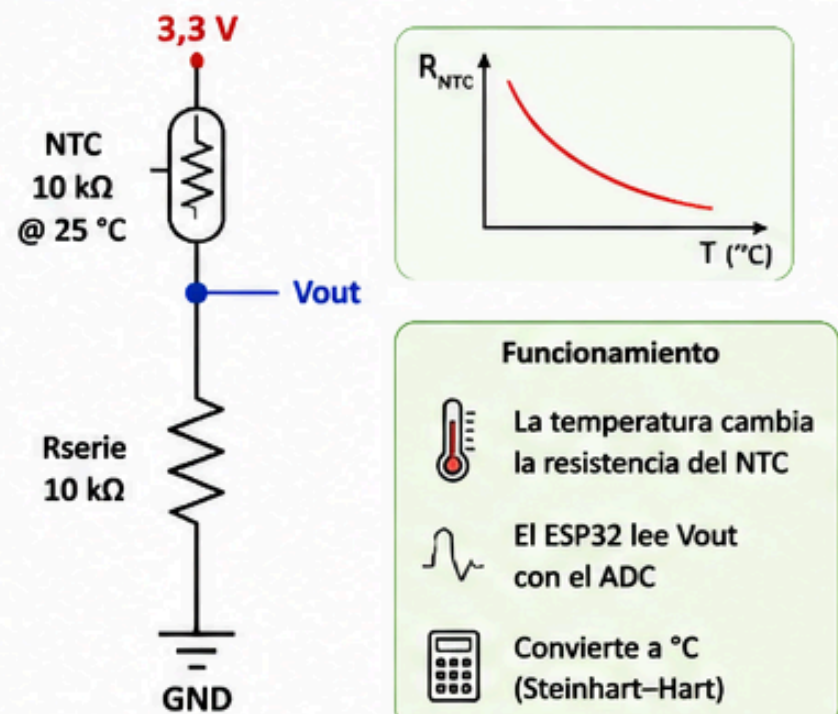


Etapa de potencia AC

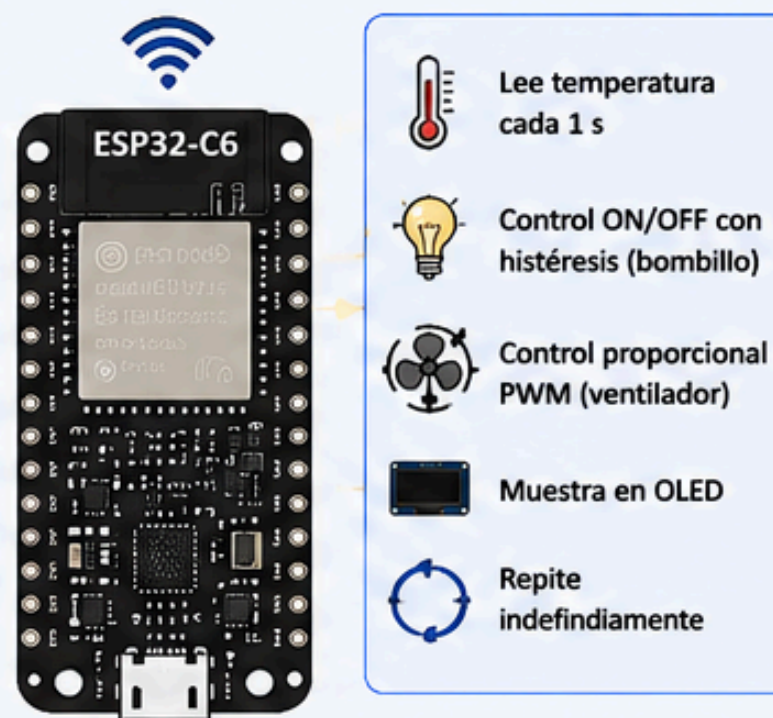


CIRCUITOS Y FUNCIONAMIENTO DEL PROYECTO

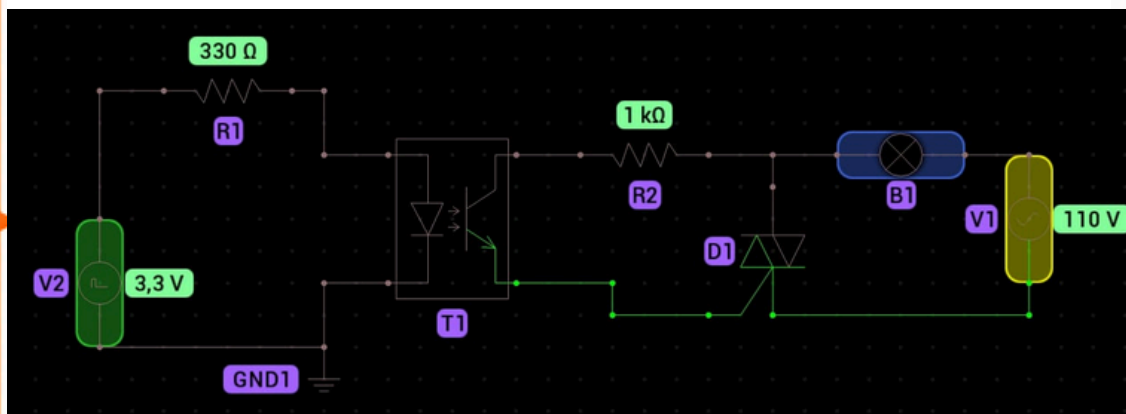
1. MEDICIÓN – TERMISTOR NTC



CONTROLADOR – ESP32-C6



2. CALEFACCIÓN – BOMBILLO AC



Funcionamiento

- El GPIO5 activa el MOC3021
- Dispara el TRIAC
- Enciende o apaga el bombillo

Control con histéresis

- ON si $T \leq 24,0\text{ °C}$
- OFF si $T \geq 25,0\text{ °C}$
- Mantiene estado entre $24,0\text{ °C}$ y $25,0\text{ °C}$

3. REFRIGERACIÓN – VENTILADOR DC (PWM)



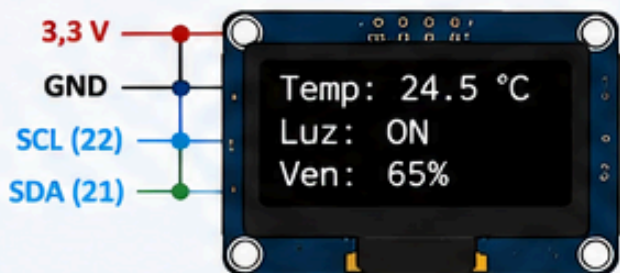
Control proporcional PWM



5. ALIMENTACIONES



4. PANTALLA OLED SSD1306 (I2C)



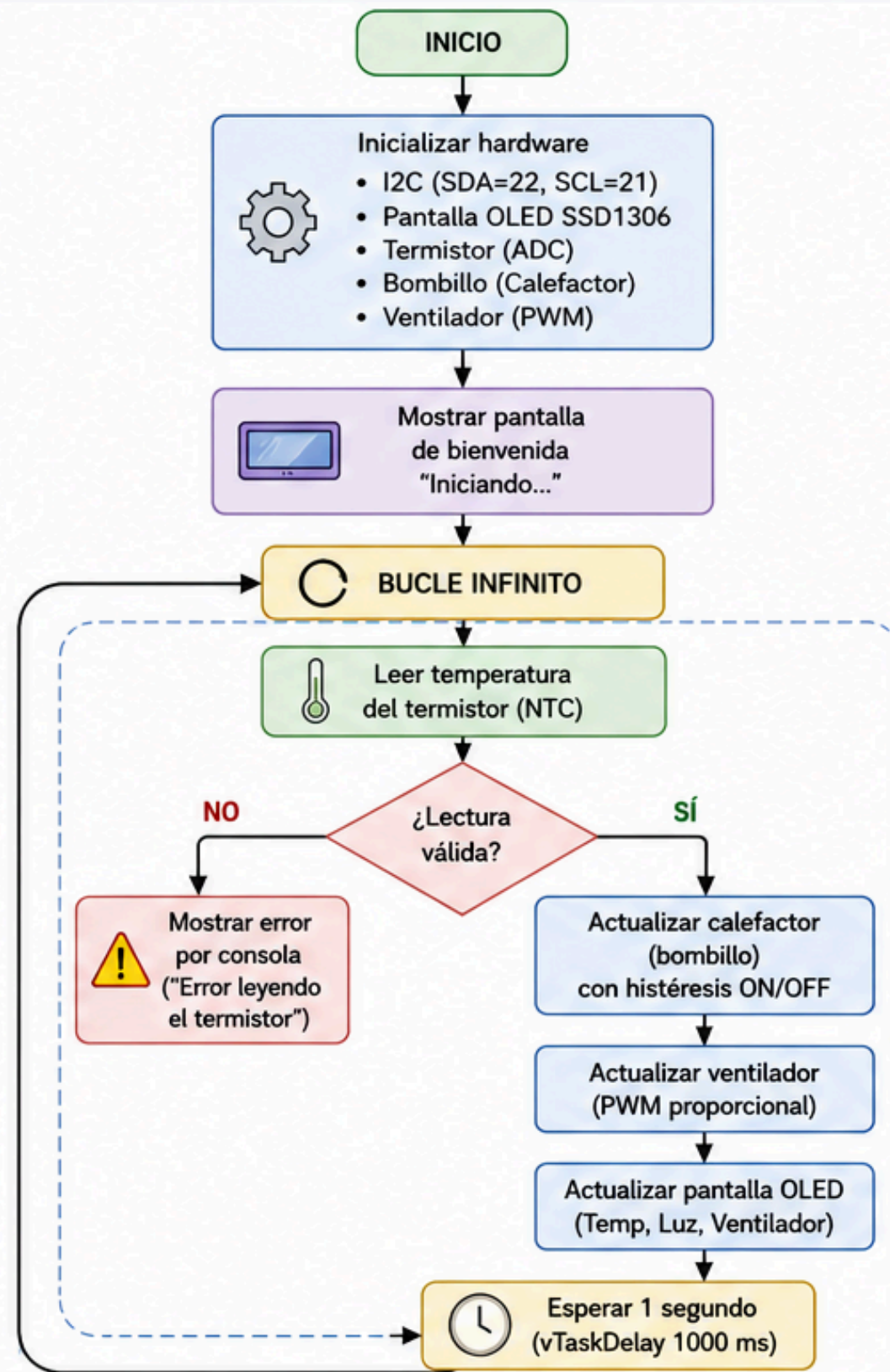
Muestra en tiempo real



FUNCIONAMIENTO GENERAL DEL SISTEMA



1. DIAGRAMA DE FLUJO GENERAL DEL SISTEMA



¿QUÉ HACE EL BUCLE INFINITO?

El bucle infinito mantiene el sistema funcionando continuamente.

Cada 1 segundo (período de muestreo):

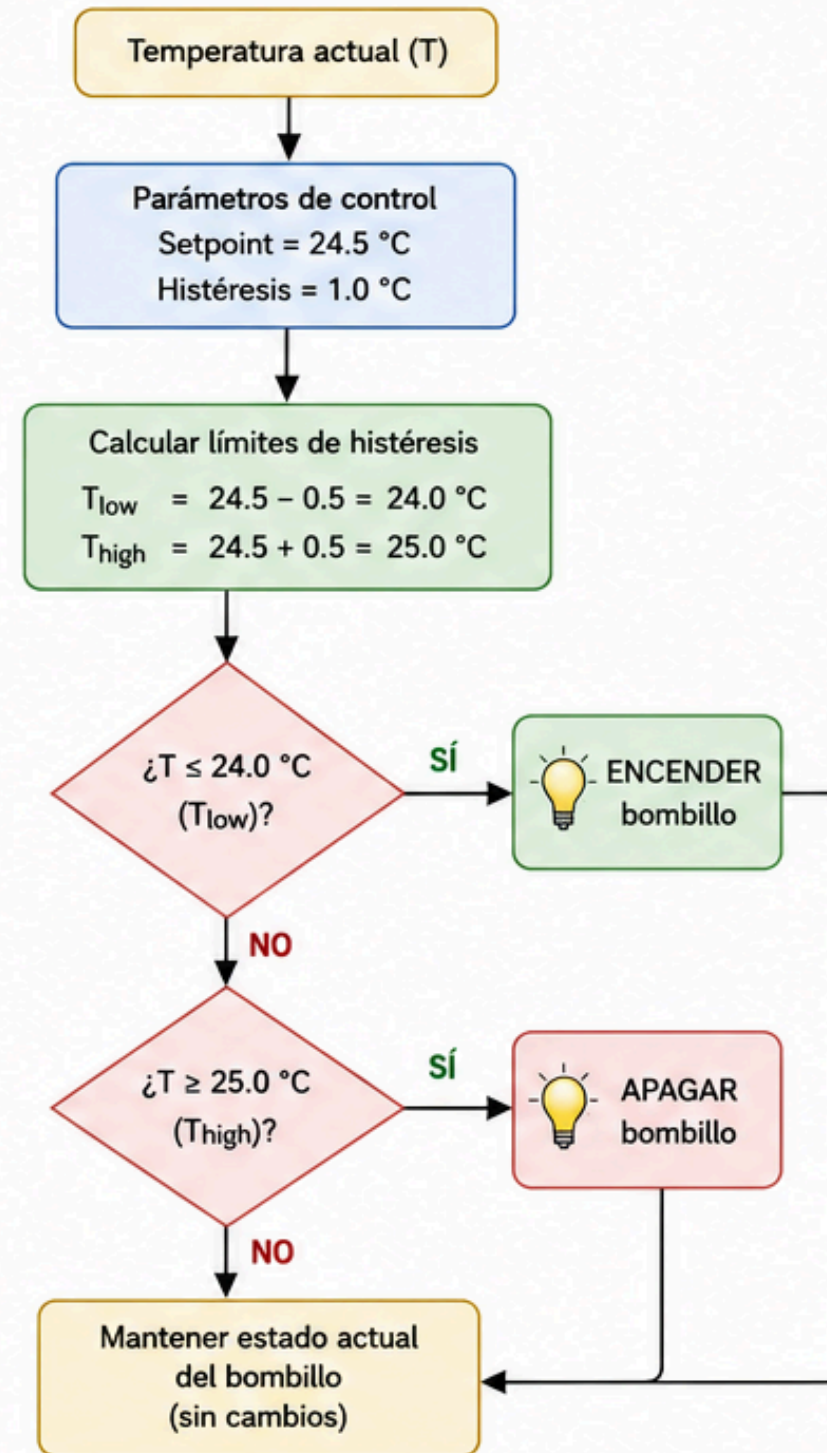
- Lee la temperatura del termistor
- Ejecuta el control del bombillo y del ventilador
- Actualiza la información en la pantalla OLED
- Muestra los datos por consola

y repite el proceso indefinidamente mientras el sistema esté encendido.



Período de muestreo:
1 segundo
(1 Hz)

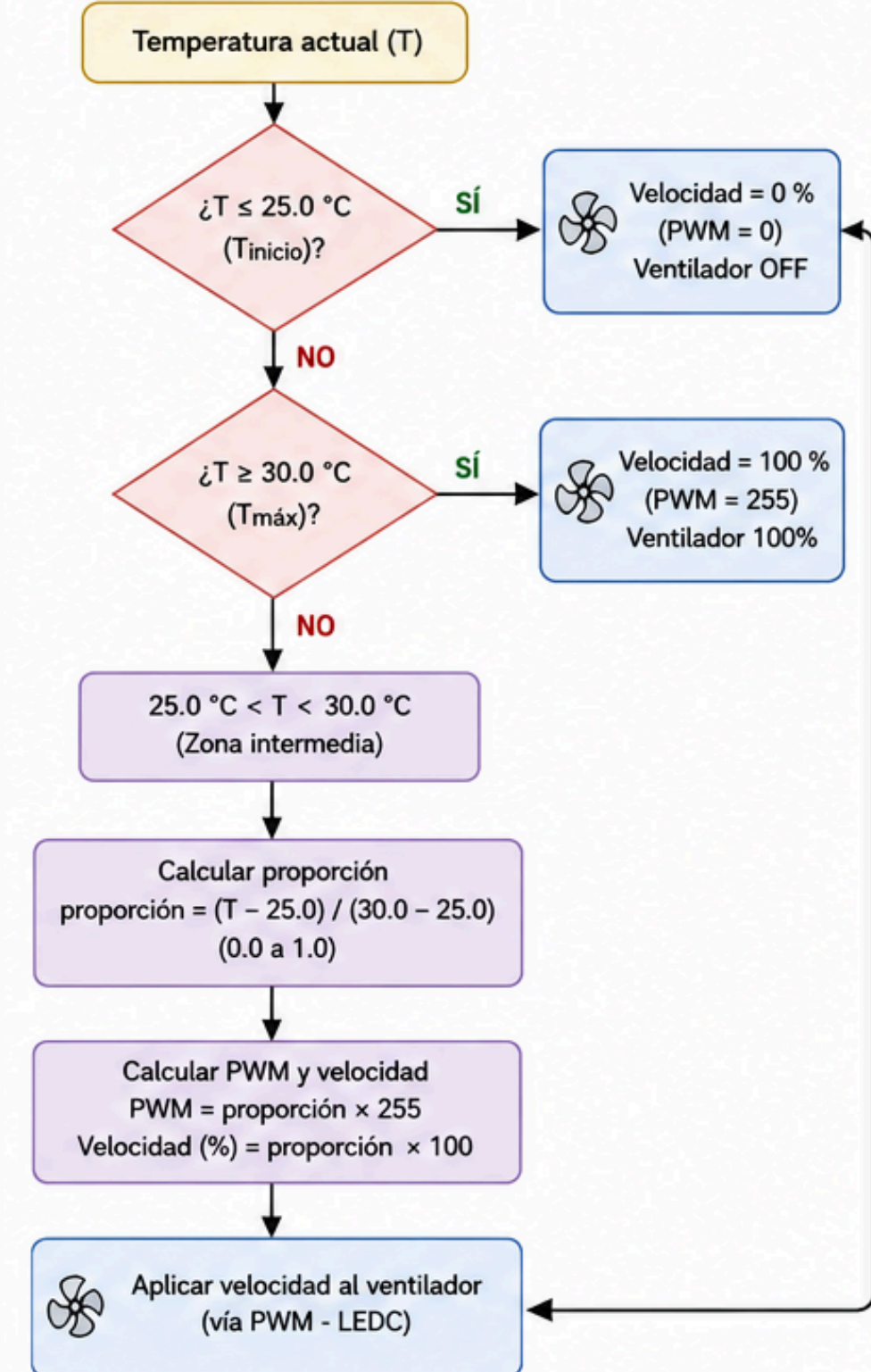
2. LÓGICA DEL CALEFACTOR (BOMBILLO) CONTROL ON/OFF CON HISTÉRESIS



El bombillo (calefactor) utiliza control ON/OFF con histéresis para evitar conmutaciones rápidas alrededor del setpoint.

- Se enciende cuando la temperatura baja de 24.0 °C
- Se apaga cuando la temperatura sube de 25.0 °C
- Mantiene el estado en la zona intermedia (24.0 – 25.0 °C)

3. LÓGICA DEL VENTILADOR CONTROL PROPORCIONAL PWM



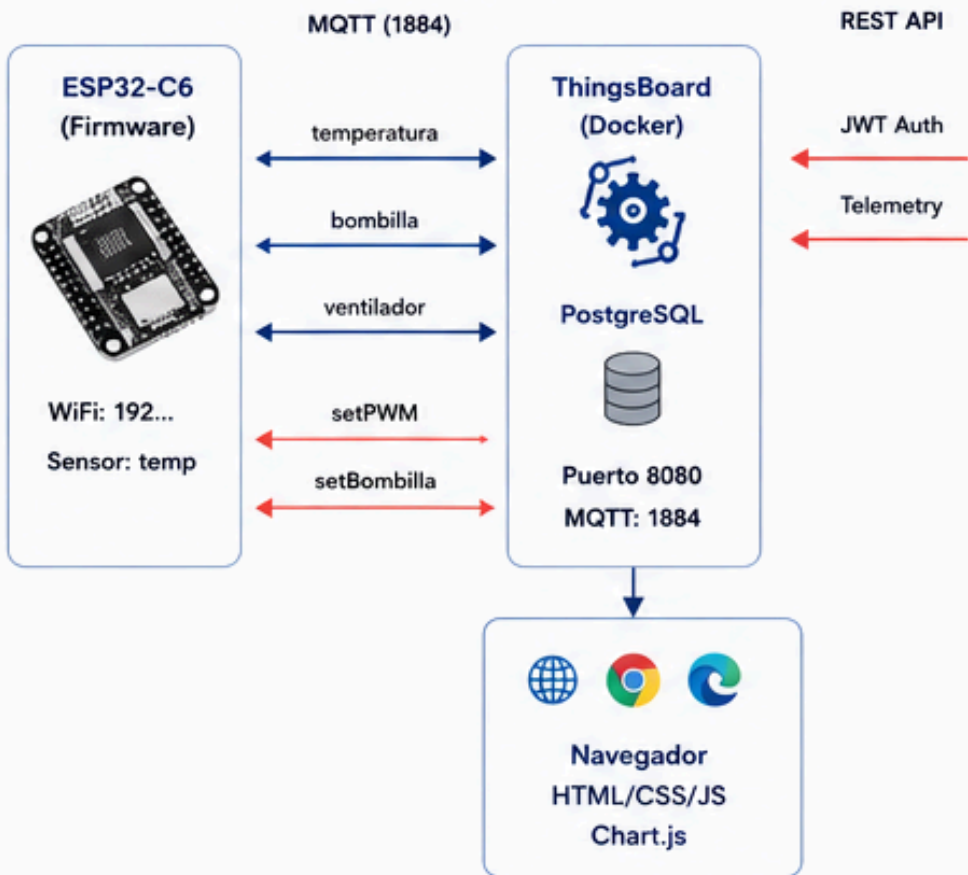
El ventilador usa control proporcional PWM:

- $\leq 25.0\text{ °C}$ → 0% (apagado)
- $\geq 30.0\text{ °C}$ → 100% (máxima potencia)
- Entre 25.0 °C y 30.0 °C → velocidad proporcional

Implementación de ThingsBoard en el Proyecto

Resumen en 5 pasos

1 Arquitectura del Sistema



2 Configuración de ThingsBoard con Docker

docker-compose.yml

```
services:
  thingsboard:
    image: thingsboard/tb-postgres
    restart: always
    ports:
      - "8080:9090" # UI web
      - "1884:1883" # MQTT
      - "8883:8883" # MQTT SSL
      - "7070:7070" # Edge
    environment:
      TB_QUEUE_TYPE: in-memory
    volumes:
      - tb-data:/data
      - tb-logs:/var/log/thingsboard
  volumes:
    tb-data:
    tb-logs:
```

Comandos Docker

```
# Iniciar
docker compose up -d
# Ver estado
docker compose ps
# Ver logs
docker compose logs -f thingsboard
# Detener
docker compose down
```

3 Comunicación ESP32 ↔ ThingsBoard vía MQTT

Flujo de Conexión

- 1 Inicializa NVS, WiFi, hardware (2C, ADC, GPIO, PWM)
- 2 Conecta a WiFi: "PEDRO VASQUEZ 1" (DOBLE ESPACIO)
IP obtenida por DHCP (ej: 192.168.1.38)
- 3 Crea cliente MQTT y conecta a 192.168.1.25:1884
Autentica con token: "MnCIBDPkA3jXFXS8iVv"
- 4 Suscribe a RPC:
Topic: v1/devices/me/rpc/request/+
Métodos: setPWM, setBombilla
- 5 Bucle principal (cada 50ms):
 - tb.loop() → procesa mensajes MQTT entrantes
 - Cada 500ms: lee temperatura del termistor actualiza bombilla y ventilador
 - Cada 1s: envía telemetría empaquetada: temperatura (float), bombilla (0/1), ventilador (0-100%)

Envío de Telemetría (MQTT)

```
// Enviar telemetría cada 2 ciclos (1s)
if (++telemetry_tick % 2 == 0) {
  telemetry_tick = 0;
  int bomb_state = ctx->bombillo.is_on ? 1 : 0;
  int ven_speed = ctx->ventilador.current_speed_pct;
  tb.sendTelemetryData(TEMPERATURE_KEY, temperatura);
  tb.sendTelemetryData(BOMBILLA_KEY, bomb_state);
  tb.sendTelemetryData(VENTILADOR_KEY, ven_speed);
}
```

RPC Bidireccional (ThingsBoard → ESP32)

```
// Callback: setPWM(int 0..255)
static void rpc_setPWM(const JsonVariantConst &data,
  JsonDocument &response) {
  int pwm_value = data.as<int>();
  ledc_set_duty(LED_CHANNEL_0, LEDC_CHANNEL_0, pwm_value);
  ledc_update_duty(LED_CHANNEL_0, LEDC_CHANNEL_0);
}
// Callback: setBombilla(bool)
static void rpc_setBombilla(const JsonVariantConst &data,
  JsonDocument &response) {
  bool state = data.as<bool>();
  bomb_set_state(&ctx->bombillo, state);
}
```

Protocolo MQTT de ThingsBoard

Dirección	Topic	Payload
ESP32 → TB	v1/devices/me/telemetry	{ "key": value }
TB → ESP32	v1/devices/me/rpc/request/{id}	{ "method": "setPWM", "params": 128 }
ESP32 → TB	v1/devices/me/rpc/response/{id}	{ "result": "ok", "pwm": 128 }
ESP32 → TB	v1/devices/me/attributes	Atributos del dispositivo

4 Dashboard Web Full-Stack

Estructura del Proyecto

```
C:\Users\everh\thingsboard\dashboard\
├── server.py          # Backend FastAPI
├── requirements.txt   # Dependencias Python
├── iniciar.bat        # Script de arranque
├── detener.bat        # Script de parada
├── static/
│   ├── index.html     # Frontend SPA
│   ├── css/
│   │   └── dashboard.css # Estilos (tema oscuro)
│   └── js/
│       ├── chart.umd.min.js # Chart.js (local, no CDN)
│       ├── api.js        # Cliente de API modular
│       └── dashboard.js   # Lógica del dashboard
```

Backend (FastAPI) — server.py

Endpoint	Método	Función
/api/health	GET	Verifica conexión con ThingsBoard
/api/device/info	GET	Info del dispositivo ESP32
/api/telemetry/latest?keys=	GET	Último valor de telemetría
/api/telemetry/history?keys=&startTs=&endTs=	GET	Histórico de telemetría
/api/rpc/{method}	POST	Enviar comando RPC al ESP32

Instalación de Dependencias

```
pip install fastapi uvicorn httpx python-multipart
```

Flujo de autenticación del backend

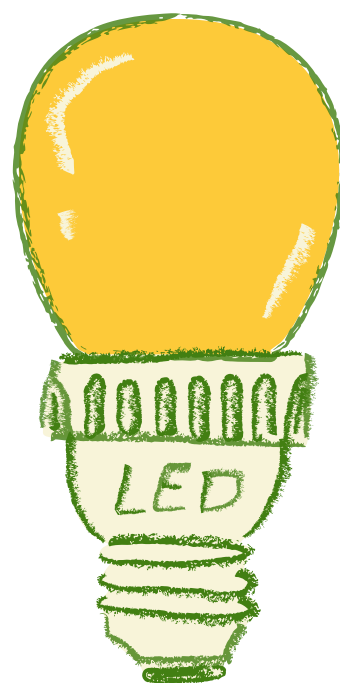
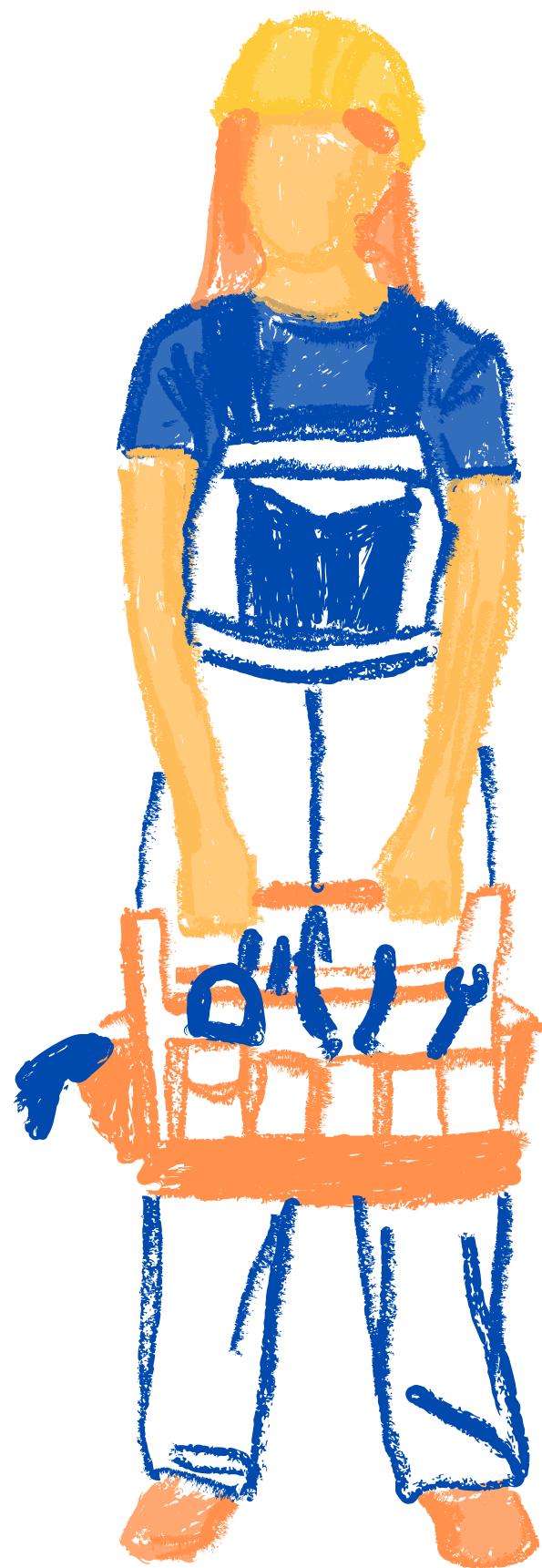
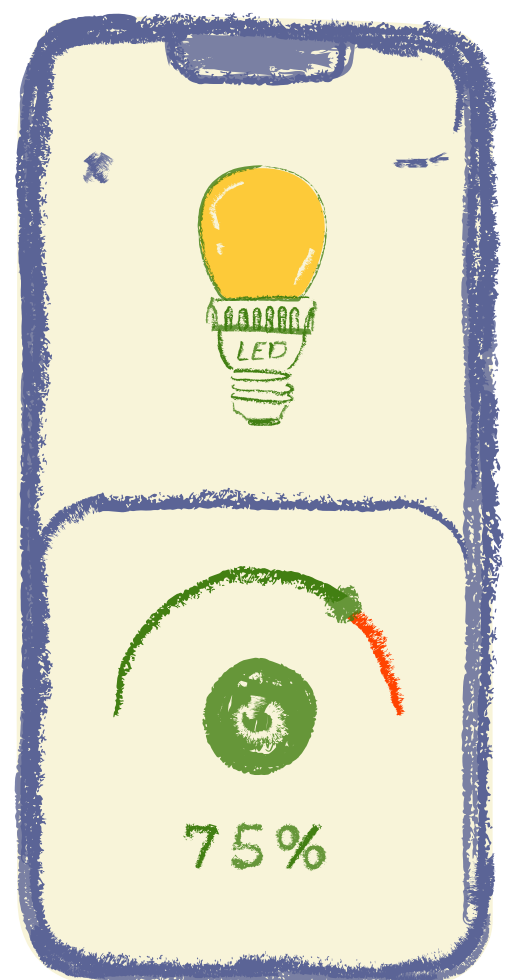
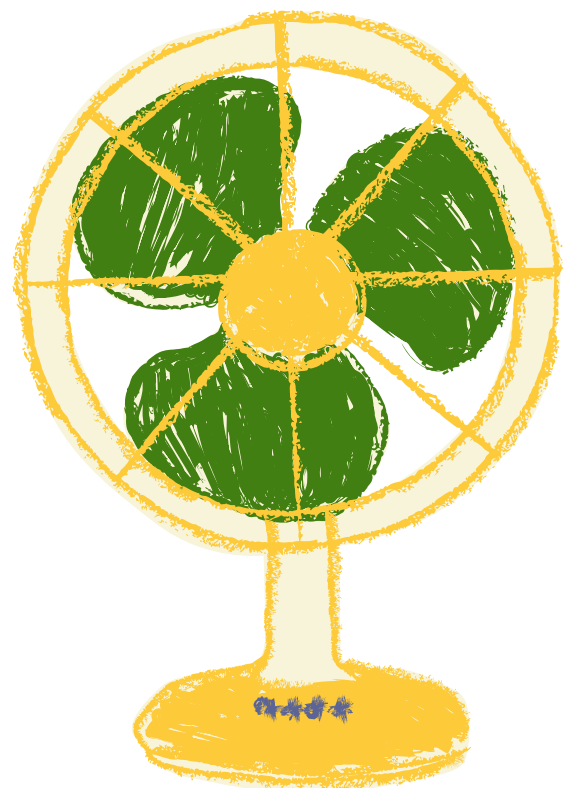
1. Obtiene JWT (POST /api/auth/login)
2. Cachea el JWT en memoria
3. Reusa el JWT en todas las llamadas
4. Se refresca cuando expira

5 Problemas Encontrados y Soluciones

#	Problema	Causa	Solución
1	Conflicto de Puerto MQTT 1883 con Mosquitto	Mosquitto ocupaba el puerto 1883. El ESP32 se conectaba a Mosquitto.	Cambiar mapeo de puertos en Docker de 1883:1883 a 1884:1883 y usar puerto 1884 en el ESP32.
2	SSID WiFi con Doble Espacio	SSID real tenía DOBLE espacio entre "VASQUEZ" y "1".	En main.cpp: "PEDRO VASQUEZ 1" (con doble espacio).
3	Tabla queue no existe en PostgreSQL	Faltaba ejecutar instalación/migración de esquema en tb-node.	Usar imagen thingsboard/tb-postgres (monolítica con PostgreSQL embebido).
4	Token de Acceso ya Asignado	Ya existía el dispositivo con ese token.	Usar el dispositivo existente.
5	Valores Booleanos en Telemetría	Enviar bool (true/false) generaba error 500.	Cambiar tipo de dato a int (0/1): int bomb_state = ctx->bombillo.is_on ? 1 : 0;
6	Dashboard mostraba "Desconectado"	Promise.all() fallaba si una request tenía problemas.	Separar llamadas en try-catch independientes.
7	Chart.js desde CDN no Cargaba	No cargaba desde cdn.jsdelivr.net	Descargar Chart.js localmente en /static/js/.
8	Puerto COM12 Ocupado al Flashear	Proceso Python mantenía el puerto abierto.	Cerrar todos los procesos Python antes de flashear: Stop-Process -Name "python" -Force -ErrorAction SilentlyContinue
9	La IP de la Máquina Cambiaba entre Redes	La IP de la PC cambiaba entre 172.20.25.9 y 192.168.1.25.	Verificar IP con Get-NetIPAddress -InterfaceAlias "Wi-Fi" y actualizar THINGSBOARD_SERVER si es necesario.

Repositorio GitHub

Repositorio GitHub



Gracias!